

Booth's Algorithm for Binary Multiplication

Computer Architecture and Organization Notes

June 27, 2025

1 Introduction

Booth's algorithm, developed by Andrew Donald Booth in 1951, is an efficient method for multiplying two signed binary numbers. It is particularly useful in computer architecture because it reduces the number of arithmetic operations required compared to traditional multiplication methods.

2 Key Concepts

2.1 Purpose

- Multiply two signed binary numbers efficiently
- Reduce the number of addition/subtraction operations
- Handle both positive and negative numbers using 2's complement representation
- Optimize hardware implementation in processors

2.2 Basic Principle

Booth's algorithm is based on the observation that multiplication by a sequence of 1's can be replaced by:

- A subtraction at the beginning of the sequence
- An addition at the end of the sequence

For example: $3 \times 7 = 3 \times (8 - 1) = 3 \times 8 - 3 \times 1$

3 Algorithm Components

3.1 Registers Required

- **A (Accumulator):** Initially 0, stores the upper bits of the result
- **Q (Multiplier):** Initially contains the multiplier

- **M (Multiplicand):** Contains the multiplicand throughout
- **Q₋₁:** Extra bit, initially 0
- **Counter:** Tracks the number of iterations (n bits for n-bit numbers)

3.2 Operations

- **Arithmetic Right Shift:** Shifts A, Q, and Q₋₁ right by 1 position
- **Addition:** $A = A + M$
- **Subtraction:** $A = A - M$ (or $A = A + 2$'s complement of M)

4 Booth's Algorithm Steps

1. Initialize:
 - $A = 0$ (n bits)
 - $Q = \text{Multiplier}$ (n bits)
 - $M = \text{Multiplicand}$ (n bits)
 - $Q_{-1} = 0$
 - $\text{Counter} = n$
2. While Counter $\neq 0$:
 - Examine Q_0 and Q_{-1} :
 - If $Q_0Q_{-1} = 00$: No operation, just shift
 - If $Q_0Q_{-1} = 01$: $A = A + M$
 - If $Q_0Q_{-1} = 10$: $A = A - M$
 - If $Q_0Q_{-1} = 11$: No operation, just shift
 - Perform arithmetic right shift on AQQ_{-1}
 - Decrement Counter
3. Final result is in AQ (A contains upper bits, Q contains lower bits)

5 Decision Table

Q ₀	Q ₋₁	Operation
0	0	Shift only
0	1	Add M to A, then shift
1	0	Subtract M from A, then shift
1	1	Shift only

6 Example: $3 \times (-4)$ using 4-bit representation

Given:

- Multiplicand (M) = 3 = 0011
- Multiplier (Q) = -4 = 1100 (2's complement)
- Expected result = -12 = 10110100 (8-bit)

Initial Setup:

- A = 0000, Q = 1100, Q_{-1} = 0, M = 0011
- Counter = 4

Step	A	Q	Q_{-1}	Q_0Q_{-1}	Operation	Counter
Initial	0000	1100	0	00	-	4
1	0000 0000	1100 0110	0 0	00	Shift only	3
2	0000 0000	0110 0011	0 0	00	Shift only	2
3	0000 1101 1110	0011 0011 1001	0 0 1	10	A = A - M Shift	1
4	1110 1111	1001 0100	1 1	11	Shift only	0

Final Result: AQ = 11110100 = -12 (correct!)

7 Example: $(-3) \times 4$ using 4-bit representation

Given:

- Multiplicand (M) = -3 = 1101 (2's complement)
- Multiplier (Q) = 4 = 0100
- Expected result = -12 = 10110100 (8-bit)

Initial Setup:

- A = 0000, Q = 0100, Q_{-1} = 0, M = 1101
- Counter = 4

Step	A	Q	Q ₋₁	Q ₀ Q ₋₁	Operation	Counter
Initial	0000	0100	0	00	-	4
1	0000	0100	0	00	Shift only	3
	0000	0010	0			
2	0000	0010	0	00	Shift only	2
	0000	0001	0			
3	0000	0001	0	10	A = A - M Shift	1
	0011	0001	0			
	0001	1000	1			
4	0001	1000	1	01	A = A + M Shift	0
	1110	1000	1			
	1111	0100	0			

Final Result: AQ = 11110100 = -12 (correct!)

8 Advantages of Booth's Algorithm

- **Efficiency:** Reduces the number of arithmetic operations, especially for numbers with consecutive 1's
- **Signed Multiplication:** Handles both positive and negative numbers naturally using 2's complement
- **Hardware Implementation:** Suitable for hardware implementation in processors
- **Uniform Operation:** Same algorithm works for all combinations of positive and negative numbers
- **Speed:** Faster than traditional add-and-shift multiplication for certain bit patterns

9 Disadvantages of Booth's Algorithm

- **Complexity:** More complex to understand and implement than simple multiplication
- **Worst Case:** No improvement for alternating bit patterns (010101...)
- **Hardware Requirements:** Requires additional logic for decision making
- **Sequential Nature:** Cannot be easily parallelized

10 Applications

- **Computer Processors:** Used in ALUs for efficient multiplication
- **Digital Signal Processing:** Important for real-time signal processing applications
- **Embedded Systems:** Useful where computational efficiency is critical
- **Cryptography:** Used in modular arithmetic operations

11 Variations and Extensions

11.1 Modified Booth Algorithm (Radix-4)

- Examines 3 bits at a time instead of 2
- Can reduce the number of iterations by half
- More complex decision logic but faster execution

11.2 Higher Radix Booth Algorithms

- Radix-8, Radix-16 algorithms exist
- Trade complexity for speed
- Used in high-performance processors

12 Validity of Booth's Multiplication Algorithm

12.1 Mathematical Foundation

The validity of Booth's algorithm can be proven by showing that it correctly implements the mathematical definition of multiplication for signed binary numbers in 2's complement representation.

12.2 Key Observation

Consider a binary number with a sequence of consecutive 1's:

$$\dots 01111\dots 10\dots$$

This can be rewritten as:

$$\dots 10000\dots 0 - \dots 00000\dots 10\dots$$

In other words: $2^{j+1} - 2^i$ where the 1's span from bit position i to bit position j .

12.3 Formal Proof

Let M be the multiplicand and Q be the multiplier. The multiplier Q can be written as:

$$Q = \sum_{i=0}^{n-1} q_i \cdot 2^i$$

where q_i is the i -th bit of Q .

For any sequence of consecutive 1's in Q from position i to j , we have:

$$\sum_{k=i}^j 2^k = 2^{j+1} - 2^i$$

Therefore, instead of performing $(j - i + 1)$ additions, we can:

- Subtract $M \cdot 2^i$ (at the beginning of the sequence)
- Add $M \cdot 2^{j+1}$ (at the end of the sequence)

This reduces the number of operations from $(j - i + 1)$ to just 2 operations.

12.4 Algorithm Correctness

The algorithm examines pairs of bits (q_i, q_{i-1}) and performs operations based on the following logic:

1. **Case 00 or 11:** No transition occurs
 - No addition or subtraction needed
 - Just perform the shift operation
2. **Case 01:** Transition from 0 to 1 (beginning of a sequence of 1's)
 - Add the multiplicand: $A = A + M$
 - This handles the $+2^{j+1}$ term
3. **Case 10:** Transition from 1 to 0 (end of a sequence of 1's)
 - Subtract the multiplicand: $A = A - M$
 - This handles the -2^i term

12.5 Proof by Induction

Base Case: For a single bit multiplication, the algorithm correctly handles all four cases (00, 01, 10, 11).

Inductive Step: Assume the algorithm works correctly for $(n - 1)$ bits. For the n -th bit:

- The algorithm examines bits (q_n, q_{n-1})
- Based on the transition, it performs the appropriate operation
- The shift operation correctly positions the result for the next iteration
- The final result is the sum of all partial products

12.6 Example Verification

Consider the multiplication $3 \times 6 = 18$:

- $M = 3 = 0011_2$
- $Q = 6 = 0110_2$
- Traditional method: $3 \times (2^1 + 2^2) = 3 \times 2 + 3 \times 4 = 6 + 12 = 18$
- Booth's method: $3 \times (2^3 - 2^1) = 3 \times 8 - 3 \times 2 = 24 - 6 = 18$

Both methods yield the same result, confirming the algorithm's correctness.

12.7 Handling Negative Numbers

For negative numbers in 2's complement:

- The most significant bit (MSB) has a weight of -2^{n-1}
- Booth's algorithm naturally handles this through the arithmetic right shift
- The sign bit is preserved during shifting, maintaining the correct value

12.8 Invariant Property

Throughout the algorithm execution, the following invariant holds:

$$M \times Q_{original} = A \times 2^{steps} + Q_{current}$$

where:

- $Q_{original}$ is the initial multiplier
- A is the current accumulator value
- $Q_{current}$ is the current multiplier value
- $steps$ is the number of steps completed

This invariant ensures that the final result $A \times 2^n + Q_{final}$ equals $M \times Q_{original}$.

12.9 Conclusion

The validity of Booth's algorithm is established through:

1. Mathematical equivalence of bit sequence representation
2. Correct handling of all possible bit transitions
3. Preservation of the multiplication invariant
4. Proper treatment of signed numbers in 2's complement

The algorithm's correctness is independent of the specific bit patterns in the operands, making it a robust and reliable multiplication technique for all signed binary numbers.

13 Summary

Booth's algorithm is a fundamental technique in computer arithmetic that provides an efficient method for binary multiplication of signed numbers. While it may not always provide speedup compared to simple multiplication, its ability to handle signed numbers uniformly and its suitability for hardware implementation make it an important algorithm in computer architecture.

The key insight of Booth's algorithm is recognizing that multiplication by a string of 1's can be optimized by using subtraction and addition at the boundaries, leading to fewer arithmetic operations in many practical cases.

14 Array Implementation of the Booth Multiplication Algorithm

14.1 Overview

The array implementation of Booth's algorithm provides a hardware-efficient way to perform multiplication using a systematic arrangement of processing cells. This implementation is particularly useful for VLSI design and high-speed multiplication units.

14.2 Multifunction Cell (B Cell)

The array implementation requires a multifunction cell capable of three operations:

- Addition
- Subtraction
- No operation (skip)

14.3 B Cell Logic Functions

The functions of the B cell are defined by the following equations:

Sum Output:

$$z = a \oplus bH \oplus cH$$

Carry Output:

$$C_{out} = (a \oplus D)(b + c) + bc$$

where:

- a = input bit from the multiplicand
- b = carry input from previous stage
- c = sum input from previous stage
- D = control signal for operation type
- H = control signal for cell activation

14.4 Operation Modes

The B cell operates in different modes based on the control signals H and D :

14.4.1 Full Adder Mode

When $HD = 10$, the equations reduce to full adder equations:

$$z = a \oplus b \oplus c \tag{1}$$

$$C_{out} = ab + ac + bc \tag{2}$$

14.4.2 Full Subtractor Mode

When $HD = 11$, the equations reduce to full subtractor equations:

$$z = a \oplus b \oplus c \quad (3)$$

$$C_{out} = \bar{a}b + \bar{a}c + bc \quad (4)$$

14.4.3 Skip Mode

When $H = 0$, then:

$$z = a \quad (5)$$

$$C_{out} = \text{carry plays no role in the final result} \quad (6)$$

14.5 Array Structure

A $n \times n$ bit multiplier is constructed from:

$$n^2 + \frac{n(n-1)}{2} \text{ cells}$$

This includes:

- n^2 B cells for multiplication operations
- $\frac{n(n-1)}{2}$ additional cells for carry propagation

14.6 Control Cell (C Cell)

The C cell generates control inputs H and D required by the B cells depending on the combination of $x_i x_{i-1}$.

14.6.1 Control Truth Table

x_i	x_{i-1}	H	D
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	0

14.6.2 Control Logic Interpretation

- $x_i x_{i-1} = 00$: No operation (skip), $H = 0, D = 0$
- $x_i x_{i-1} = 01$: Addition required, $H = 1, D = 0$
- $x_i x_{i-1} = 10$: Subtraction required, $H = 1, D = 1$
- $x_i x_{i-1} = 11$: No operation (skip), $H = 0, D = 0$

14.7 Array Architecture

14.7.1 Cell Arrangement

The array multiplier consists of:

1. **Top row:** C cells for generating control signals
2. **Main array:** B cells arranged in a rectangular pattern
3. **Interconnections:** Data paths and control signal distribution

14.7.2 Data Flow

- Multiplicand bits flow horizontally across rows
- Partial products flow diagonally downward
- Carry signals propagate between adjacent cells
- Control signals from C cells govern B cell operations

14.8 Implementation Advantages

- **Regular Structure:** Suitable for VLSI implementation
- **Pipeline Capability:** Can be pipelined for higher throughput
- **Scalability:** Easy to extend to larger word sizes
- **Parallel Processing:** Multiple operations occur simultaneously

14.9 Hardware Complexity

For an $n \times n$ bit multiplier:

- **Total cells:** $n^2 + \frac{n(n-1)}{2}$
- **Control cells:** n C cells
- **Processing cells:** $n^2 + \frac{n(n-1)}{2} - n$ B cells
- **Delay:** Proportional to $2n - 1$ gate delays

14.10 Design Considerations

- **Cell Design:** Optimize B and C cells for speed and area
- **Layout:** Minimize interconnection delays
- **Power Consumption:** Consider dynamic power in switching cells
- **Testing:** Ensure testability of the array structure

14.11 Comparison with Sequential Implementation

Aspect	Sequential	Array
Speed	$O(n)$ cycles	$O(1)$ cycle
Area	$O(1)$	$O(n^2)$
Power	Low	High
Complexity	Simple	Complex

The array implementation trades area and power for speed, making it suitable for high-performance applications where multiplication throughput is critical.